

Knowledge-Based Neural Networks (KBNN) – Introduction

Network Training

Network training is a process used to adjust the weights of a neural network so that it can learn from data and make accurate predictions. It involves multiple steps, including weight initialization, activation calculations, clustering, and backpropagation. Below is a simplified and detailed explanation of the key concepts from the provided text.

1. Introduction to Network Training

When designing a neural network, one major challenge is ensuring it generalizes well to new data and does not overfit. Overfitting occurs when a network memorizes training data instead of learning meaningful patterns. To prevent this, various training techniques are used.

One common method is the **backpropagation algorithm**, which adjusts network weights based on the error between predicted and actual values. Another approach is **clustering hidden units** to optimize network structure.

2. Steps in Network Training

A. Weight Initialization

- Weights are the numerical parameters that determine how information flows through a network.
- Proper initialization helps the network converge faster and prevents issues like vanishing gradients.
- Weights can be initialized randomly or based on domain knowledge.

B. Calculation of Activation

- Each unit (or neuron) in the network computes an output based on its inputs and weights.
- The activation function (e.g., **sigmoid function**) is used to determine the output of each neuron.

The activation level O_j of a hidden and output unit is determined by

$$O_j = F_{CF}(W_{j1}O_1, W_{j2}O_2, \dots, -\theta_j)$$

where W_{ji} 's and O_i 's are input weights and inputs to unit j , respectively, θ_j is the threshold on the unit, and the function F_{CF} is the CF-based activation function. Another alternative is

$$O_j = F(\sum W_{ji}O_i - \theta_j)$$

where F is a sigmoid function.

C. Clustering Hidden Units

- Hidden units are grouped into clusters based on similarity.
- The similarity is measured using weight vectors.
- If the similarity is above a threshold (e.g., 0.85), the units are merged; otherwise, a new cluster is formed.
- This step helps optimize the network and reduces redundancy.

Clustering procedure:

1. Start with no clusters.

2. Assign the first hidden unit to a new cluster.
3. For each remaining hidden unit:
 - Find the most similar cluster.
 - If similarity > threshold, update weight vectors.
 - Otherwise, create a new cluster.

After clustering, new hidden units are introduced with updated weight vectors.

D. Weight Training using Backpropagation

- **Backpropagation** is the most widely used training method.
- It involves adjusting weights to minimize the difference between predicted and actual outputs.
- This is done using **gradient descent**, which updates weights based on errors.

Steps in backpropagation:

1. Compute the error at the output layer:

– For output units:

$$D_j = T_j - O_j$$

where T_j is the desired (target) output activation and O_j is the actual output activation at output unit j .

– For hidden units:

$$D_j = \sum_k \left(\frac{\partial O_k}{\partial O_j} \right) D_k$$

where D_k is the discrepancy at unit k to which a connection points from hidden unit j . In the summation, each discrepancy D_k is weighted by the partial derivative of O_k with respect to O_j .

2. Update the weights:

Start at the output units and work backward to the hidden layers recursively. Adjust weights by

$$W_{ji}(t+1) = W_{ji}(t) + \Delta W_{ji}$$

where $W_{ji}(t)$ is the weight from unit i to unit j at iteration t and ΔW_{ji} is the weight adjustment.

The weight change is computed by

$$\Delta W_{ji} = \eta D_j \left(\frac{\partial O_j}{\partial W_{ji}} \right) - \zeta W_{ji}(t) \quad (4.1)$$

where η is a trial-independent learning rate, ζ is a weight-decay parameter ($\zeta = 0$ for knowledge-based connections), D_j is the discrepancy (error) between the actual and the desired output activation at unit j , and the term $\partial O_j / \partial W_{ji}$ is the partial derivative of O_j with respect to W_{ji} . Adding a momentum term is optional.

3. Repeat the process until the stopping condition is met.

Key Features of Training

- **Weight range:** Weights typically range between -1 and 1.
- **Activation function:** The sigmoid function or other non-linear functions help networks learn complex patterns.
- **Learning rate tuning:** Choosing the right learning rate is critical for efficient training.
- **Momentum term:** Helps accelerate learning and prevent oscillations.

So, Network training involves:

1. **Initializing weights** appropriately.
2. **Computing activations** using an activation function.
3. **Clustering hidden units** to optimize the network.
4. **Using backpropagation** to adjust weights based on errors.

By following these steps, the network learns to make better predictions, reducing error and improving accuracy.

Network Revision and Issues in Knowledge-Based Neural Networks

Network Revision

Network revision in knowledge-based neural networks (KBNNs) involves refining and updating the network's structure and weights based on new data or insights. This process is crucial for maintaining the network's accuracy and relevance over time. The revision process typically includes the following steps:

1. **Mapping the Rule Base into a Neural Network:** The initial domain knowledge, often in the form of rules, is mapped into a neural network architecture. This involves assigning input units to data attributes, output units to target concepts, and hidden units to intermediate concepts.
2. **Training the Neural Network:** The network is trained using the backpropagation procedure on the training data. This step adjusts the weights of the connections to minimize the error between the network's output and the desired output.
3. **Revising the Trained Neural Network:** After training, the network is revised to improve its performance. This involves detecting and correcting semantically incorrect connections, which are connections that do not align with the intended logic of the rules.
4. **Translating the Revised Neural Network into Rules:** The revised network is translated back into a set of rules, which can be used to update the initial domain knowledge.

Semantically Incorrect Connections:

- **Definition:** Semantically incorrect connections are those where the weight (strength) of the connection does not align with the logical relationship between the nodes it connects. For example, a positive weight might indicate an excitatory connection, but if the actual relationship is inhibitory, the connection is semantically incorrect.
- **Detection:** The consistent-shift algorithm is used to detect semantically incorrect connections. This algorithm compares the weights before and after training. If the weight shift is inconsistent (i.e., the weight moves in the opposite direction of what is expected), the connection is flagged as semantically incorrect.
- **Revision:** Connections identified as semantically incorrect are either deleted or adjusted. Deletion of a connection from a conjunction unit to a disjunction unit means deleting a rule, while deletion of a connection from a disjunction unit or an input unit to a conjunction unit corresponds to generalizing a rule.

Issues in Network Revision

Several issues arise during the network revision process, particularly in the context of knowledge-based neural networks:

1. Strength of Initial Knowledge:

- **No Initial Knowledge:** If there is no initial knowledge, the network must be built from scratch using a purely data-driven approach. This involves creating a fully connected multilayer neural network and training it on available data.
- **Strong Initial Knowledge:** If the initial knowledge is strong, the network can be directly mapped from the existing rules without adding extra hidden units or connections. This simplifies the computation but requires that the initial knowledge is accurate and complete.
- **Weak or Incomplete Knowledge:** If the initial knowledge is weak or incomplete, additional hidden units and connections must be added to the network. The number of hidden units required can be indicated by the error curve during training. Poor convergence suggests that more hidden units are needed.

2. Preservation of Initial Semantics:

- **Semantics of Hidden Units:** The semantics of a hidden unit are reflected by its input and output weight patterns. During training, the initial semantics may be lost or distorted due to information redistribution. It is crucial to preserve the intended semantics as much as possible.
- **Activation Function:** The choice of activation function can affect the preservation of semantics. The CF-based activation function tends to maintain more built-in semantics than the sigmoid function, which can become less predictable when its slope gets steep.

3. Knowledge Hierarchy:

- **Intermediate-Level Abstraction:** Intermediate-level abstraction can provide advantages in reasoning and explanation. However, improper introduction of intermediate concepts can increase uncertainty and degrade system performance.
- **Consistent-Shift Algorithm:** This algorithm is used to determine which connections in the hierarchy can be deleted. It helps in identifying and removing unnecessary intermediate concepts and rules that undermine system performance.

4. Testing New Rules:

- **Acceptability of New Rules:** When new rules involving intermediate or hidden concepts are introduced, they must be tested against instances not described by such concepts. The consistent-shift algorithm can be applied to determine the acceptability of these new rules.

Network revision in knowledge-based neural networks is a complex but essential process for maintaining the accuracy and relevance of the network. It involves detecting and correcting semantically incorrect connections, preserving the initial semantics, and managing the knowledge hierarchy. The consistent-shift algorithm plays a crucial role in this process by identifying connections that need to be revised or deleted. The strength of the initial knowledge significantly affects the design and revision of the network, and the choice of activation function can influence the preservation of semantics. Proper handling of these issues ensures that the network remains effective and reliable over time.

1. Effect of Initial Knowledge on Model Design:

- If there is no initial knowledge, a fully connected multilayer neural network is used.
- If knowledge exists, extra hidden units and connections are needed to integrate it.

2. Uncertainty in Knowledge Completeness:

- It is difficult to determine if the initial knowledge is complete.
- Empirical evaluation is required, but practical constraints make this challenging.

3. Poor Convergence Due to Hidden Units:

- An inadequate number of hidden units can result in poor convergence.

4. Knowledge Hierarchy Challenges:

- Improper reasoning and incorrect abstraction levels may introduce errors.
- Overfitting and performance degradation can occur due to hierarchical depth.

5. Hidden Unit Mapping Issues:

- Some concepts may spread across multiple hidden units, making it difficult to retain semantic integrity.
 - Improper generalization can lead to reduced predictability.
6. **CF-Based Activation Function Limitations:**
- The activation function may introduce biases, affecting the adaptability of the model.

These issues highlight the complexities of using neural networks for knowledge-based learning and theory revision.

Decision Tree-Based Neural Networks

Decision tree-based neural networks are a hybrid approach that combines the interpretability of decision trees with the learning capabilities of neural networks. This approach is particularly useful when the initial knowledge can be represented as a decision tree, which is then mapped into a neural network structure.

Construction of Decision Tree-Based Neural Networks

1. Building the Decision Tree:

- **Discrete Domain:** For a discrete domain, the decision tree is constructed by selecting the best feature at each node that minimizes the entropy criterion. The entropy criterion measures the impurity or disorder in the data. The feature that results in the highest information gain (i.e., the greatest reduction in entropy) is chosen.
- **Continuous Domain:** For a continuous domain, the decision tree is constructed by selecting the best hyperplane (or plane) at each node that minimizes the entropy criterion. This involves finding the optimal split point for continuous features.

2. Mapping the Decision Tree to a Neural Network:

- **Hidden Layers:** The decision tree is mapped into a neural network with two hidden layers. The first hidden layer corresponds to the nonleaf nodes of the decision tree, and the second hidden layer corresponds to the leaf nodes.
- **Nodes and Connections:** Each nonleaf node in the decision tree becomes a node in the first hidden layer of the neural network. Each leaf node becomes a node in the second hidden layer. The connections between nodes in the neural network reflect the structure of the decision tree.

Example

Consider a decision tree with m nonleaf nodes and n leaf nodes. The corresponding neural network will have:

- **First Hidden Layer:** m nodes, each representing a nonleaf node in the decision tree.
- **Second Hidden Layer:** n nodes, each representing a leaf node in the decision tree.

The neural network is then trained using standard neural network training algorithms, such as backpropagation, to fine-tune the weights and improve the network's performance.

A decision tree for a two-dimensional input space (x_1, x_2) is given with the following decision rules:

- **Node N1:** $x_1 + x_2 \geq 5$
- **Node N2:** $x_1 \geq 2$
- **Node N3:** $x_2 \geq 3$

The regions formed are:

- **Region A:** ($N1 = 0, N2 = 1$)
- **Region B:** ($N1 = 0, N2 = 0$)
- **Region C:** ($N1 = 1, N3 = 1$)
- **Region D:** ($N1 = 1, N3 = 0$)

Class assignment (GIVEN):

- Regions **A and C** $\rightarrow$ **Class K1**
- Regions **B and D** $\rightarrow$ **Class K2**

Test input (GIVEN):

$$x=(x_1,x_2)=(3,3)$$

Step 1: Input Layer

Input vector:

$$x=(x_1,x_2) \quad x = (x_1, x_2) \quad x=(x_1,x_2)$$

Given test input:

$$x=(3,3) \quad x = (3, 3) \quad x=(3,3)$$

Step 2: First Hidden Layer – Decision Neurons

Each decision node is converted into a neuron using the standard neuron equation:

$$net_j = \sum w_i x_i - \theta_j$$

A **step activation function** is used:

$$O_j = \begin{cases} 1, & net_j \geq 0 \\ 0, & net_j < 0 \end{cases}$$

◆ Neuron N1

Decision rule (GIVEN):

$$x_1 + x_2 \geq 5$$

Convert to neuron equation:

$$net_{N1} = 1 \cdot x_1 + 1 \cdot x_2 - 5$$

Substitute input values:

$$\text{netN1}=3+3-5=1$$

Output:

$$\text{ON1}=1$$

◆ Neuron N2

Decision rule (GIVEN):

$$x1 \geq 2$$

Neuron equation:

$$\text{netN2}=1 \cdot x1 - 2$$

Substitute input:

$$\text{netN2}=3-2=1$$

Output:

$$\text{ON2}=1$$

◆ Neuron N3

Decision rule (GIVEN):

$$x2 \geq 3$$

Neuron equation:

$$\text{netN3}=1 \cdot x2 - 3$$

Substitute input:

$$\text{netN3}=3-3=0$$

Output:

$$\text{ON3}=1$$

Step 3: Second Hidden Layer – Region Neurons

Each region corresponds to a **path from root to leaf** in the decision tree.
Logical **AND** is implemented using **multiplication**.

◆ Region A

$(N1 = 0, N2 = 1)$

$$A = (1 - ON1) \cdot ON2$$

$$A = (1 - 1) \cdot 1 = 0$$

◆ Region B

$(N1 = 0, N2 = 0)$

$$B = (1 - ON1) \cdot (1 - ON2)$$

$$B = (1 - 1) \cdot (1 - 1) = 0$$

◆ Region C

$(N1 = 1, N3 = 1)$

$$C = ON1 \cdot ON3$$

$$C = 1 \cdot 1 = 1$$

◆ Region D

$(N1 = 1, N3 = 0)$

$$D = ON1 \cdot (1 - ON3)$$

$$D = 1 \cdot (1 - 1) = 0$$

Step 4: Output Layer – Class Neurons

◆ Class K1

(Regions A and C $\rightarrow$ GIVEN)

$$K1=A+C=0+1=1$$

◆ Class K2

(Regions B and D $\rightarrow$ GIVEN)

$$K2=B+D=0+0=0$$

Step 5: Final Decision Rule

Predicted Class= $\text{argmax}(K1, K2)$

Predicted Class=K1

Constraint-Based Neural Networks

Constraint-based neural networks are designed to solve constraint satisfaction problems, where the goal is to find a solution that satisfies as many given constraints as possible. These networks use semantic constraints to guide the learning process and ensure that the network's output adheres to the specified constraints.

Types of Constraints

1. Constraints Between Units:

- These constraints are encoded as weights between units in the network. The absolute values of the weights reflect the strength of the constraints. A positive weight signifies a supporting relationship, while a negative weight indicates an incompatible relationship.

2. Direct Constraints on Individual Units:

- These constraints are implemented as external, direct inputs to individual units. The external input serves to clamp the unit or modify its response. For example, an external input can force a unit to remain active (on) or inactive (off) regardless of other inputs.

3. Constraints for Prior Information or Probabilities:

- These constraints are implemented as biases on each unit. A positive bias tends to turn a unit on, while a negative bias tends to shut it off. This type of constraint is useful for incorporating prior knowledge or probabilities into the network.

Constraint Satisfaction Problem

The constraint satisfaction problem involves finding a solution that maximizes the goodness of fit (G) value, which measures the degree to which the desired constraints are satisfied. The G value is defined as:

fit (G) value as the degree to which the desired constraints are satisfied by a solution:

$$G = \sum_j \sum_i W_{ji} O_i O_j + \sum_j I_j O_j + \sum_j B_j O_j \quad (4.2)$$

where:

- W_{ji} is the weight from unit i to unit j .
- O_j is the activation level of unit j .
- I_j is the external input to unit j .

- B_j is the bias on unit j .

Goodness of Fit (GG):

- **Pairwise Constraints:** The first term represents the degree to which the pairwise constraints between any two units are satisfied. If the weight is positive, the presence of both hypotheses connected by the weight will increase the G value. If the weight is negative, the presence of either hypothesis alone will decrease the G value.
- **External Input Consistency:** The second term represents the consistency between the external input and the actual activation of each unit. The G value is increased if the external input and the activation level are of the same sign; otherwise, it is decreased.
- **Bias Influence:** The third term represents the influence of the bias on the unit. If the bias and the actual activation are consistent in their signs, the G value is increased; otherwise, it is reduced.

Energy Minimization

The constraint satisfaction problem can be framed as an energy minimization problem. The energy E of the system is defined as the negative of the G value:

$$E = -G$$

The goal is to minimize the energy E , which corresponds to maximizing the G value. The Hopfield network, a type of recurrent neural network, can be used to solve this problem. The network iteratively updates the states of its units to find a stable state that minimizes the energy.

Asynchronous Update Rule:

- To avoid the problem of units inhibiting each other simultaneously, an asynchronous update rule is used. This rule ensures that only one unit updates its state at a time, allowing the network to explore different states and converge to a solution that satisfies the constraints.

Summary

- **Decision Tree-Based Neural Networks:** These networks combine the interpretability of decision trees with the learning capabilities of neural networks. The decision tree is mapped into a neural network with two hidden layers, and the network is trained using standard algorithms like backpropagation.
- **Constraint-Based Neural Networks:** These networks solve constraint satisfaction problems by incorporating semantic constraints into the network's structure. The constraints are encoded as weights, external inputs, and biases. The goal is to maximize the goodness of fit (G) value, which measures the degree to which the constraints are satisfied. The problem can be framed as an energy minimization problem, and techniques like the Hopfield network and asynchronous update rules are used to find optimal solutions.